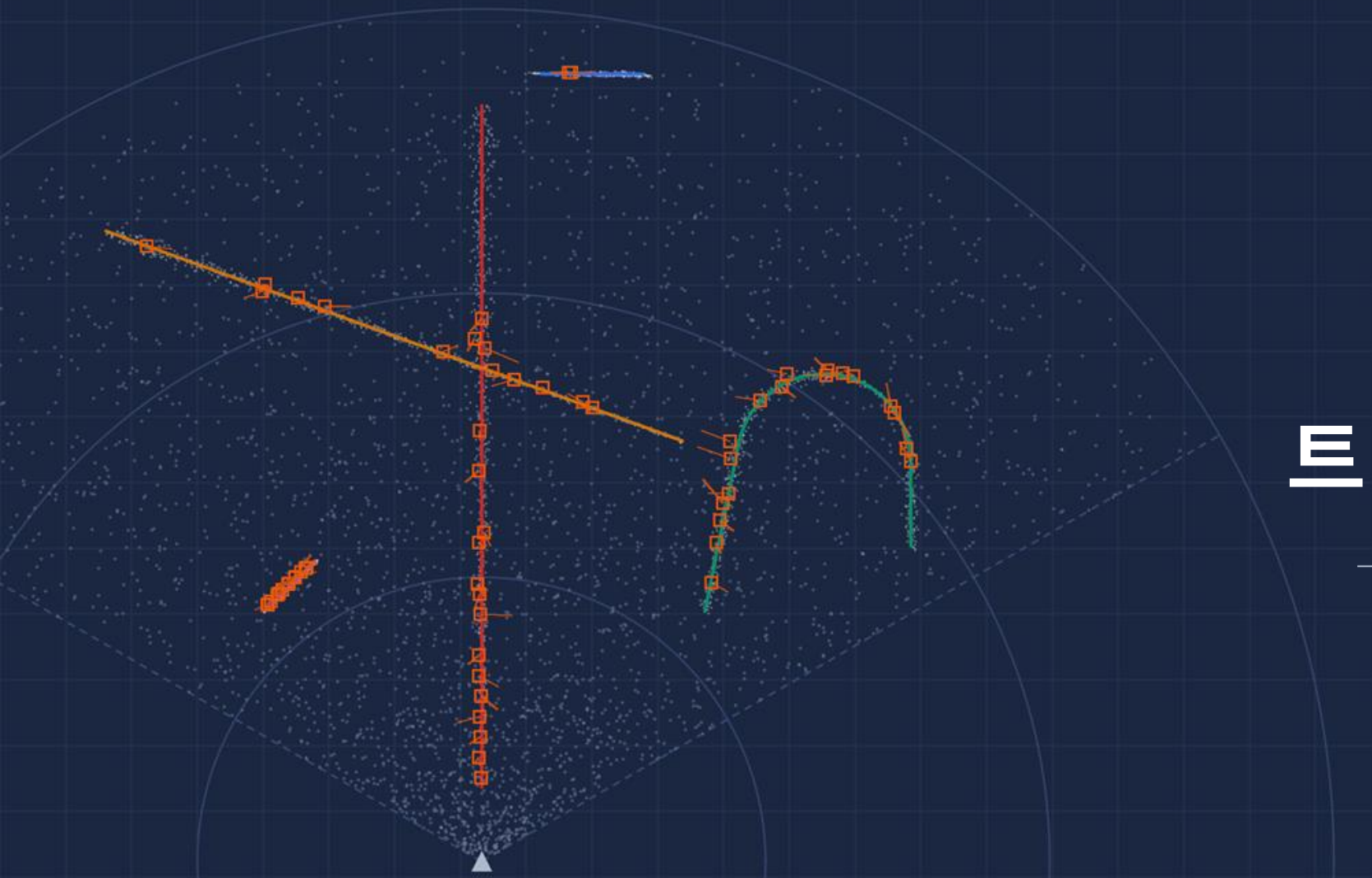
Confidential



레이다 SW 학습 과제 5

트랙 개시 · 관리

Track Initiation and Management

Janghwan Kim (Harley)

INDEX

트랙 개시 · 관리
TrackMgr

Janghwan Kim (Harley)

01 트랙 개시 3

02 트랙 관리 9

03 구현 13

04 결과 22

01. 트랙 개시

트랙이 무엇인지 보고, 플롯 두 개로 트랙을 시작하는 2 Point 개시와 속도 게이트를 설명합니다.

1.1 트랙과 트랙 개시 4

1.2 1-Point 와 2-Point 개시 5

1.3 2-Point 초기 상태변수와 오차 공분산 6

1.4 속도 게이팅 7

1.5 측정 공분산을 VC-NED 로 옮기기 8

1.1 트랙과 트랙 개시

트랙은 같은 표적의 플롯을 시간 순서로 이어 위치와 속도를 추정한 것입니다. 이번 과제는 트랙의 개시와 관리입니다.

레이다 처리 흐름에서의 위치

표적 궤적 모의
과제 3 · 시각별 참값 위치

→

측정 모의
과제 4 · 플롯 · 클러터 생성

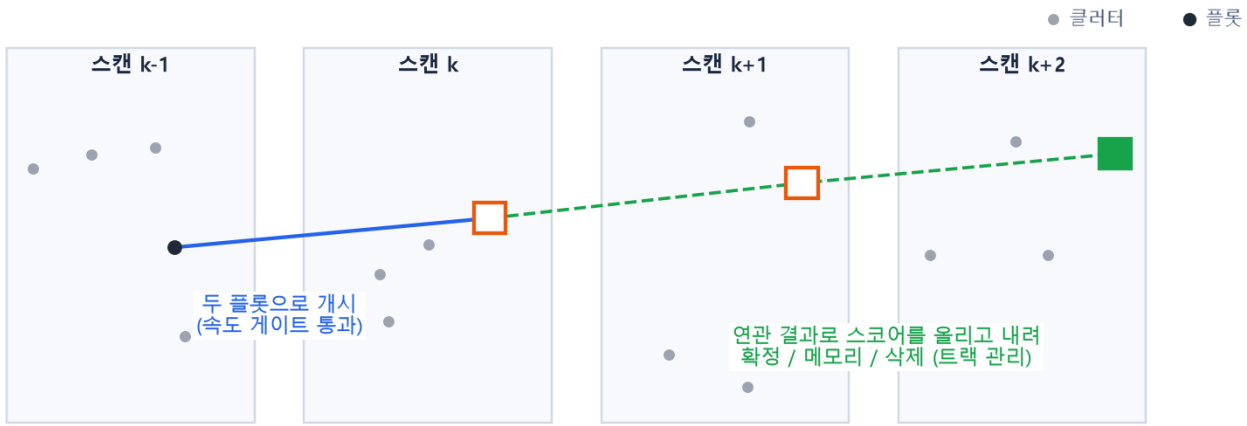
→

트랙 개시 · 관리
과제 5 · 이번 과제

→

연관 · 추적 필터
다음 과제 · 플롯을 트랙에 붙여 추정

스캔마다 들어오는 플롯과 트랙



플롯 레이다가 표적을 한 번 잰 결과. 거리 · 방위각 · 고각

클러터 표적이 아닌 것에서 나온 플롯. 같이 섞여 들어온다

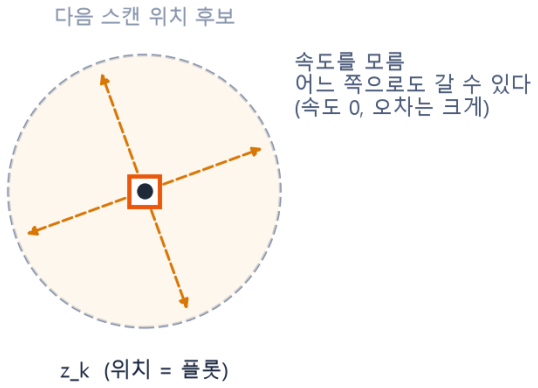
트랙 개시 같은 표적으로 보이는 플롯으로 트랙을 처음 만듦

트랙 관리 연관 결과로 확정 · 메모리 · 삭제를 정함

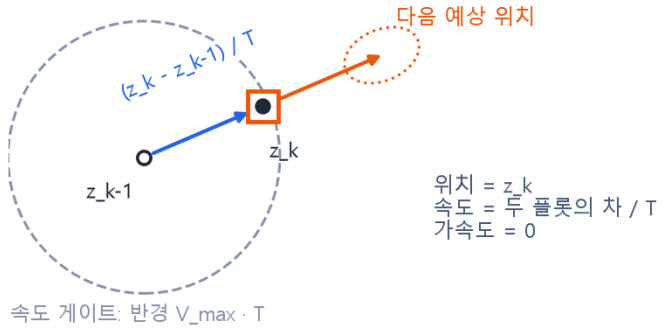
1.2 1-Point 와 2-Point 개시

플롯 하나로 바로 시작하면 속도를 모르고, 연속 두 스캔의 플롯으로 시작하면 위치 변화로 속도를 구할 수 있습니다.

1-Point: 플롯 하나로 개시



2-Point: 연속 두 스캔의 플롯으로 개시



1 **1-Point 개시**
장점: 플롯 하나로 바로 개시 (빠름)
단점: 속도를 몰라 초기 속도 오차가 크다

2 **2-Point 개시**
장점: 두 플롯의 위치 변화로 속도를 구해 1-Point 보다 속도 오차가 작다
단점: 연속 두 스캔이 필요해 개시가 늦다

3 **속도 게이트**
두 플롯이 같은 표적인지 먼저 거른다 (1.4)

이번 과제 2 Point 트랙 개시, 속도 게이트 V_max = 800 m/s, 트랙 ID 100 ~ 10000, 초기 상태변수 · 오차 공분산의 가속도 항은 0 (과제 값)

1.3 2-Point 초기 상태변수와 오차 공분산

위치는 이번 플롯, 속도는 두 플롯의 차 / T, 가속도는 0 입니다. 오차 공분산은 두 플롯의 측정 공분산 R 에서 바로 나옵니다.

상태변수 9 개 (VC-NED: x 북, y 동, z 아래)

$$X_k = [\, x \;\; x' \;\; x'' \;\; y \;\; y' \;\; y'' \;\; z \;\; z' \;\; z'' \,]^T$$

축마다 초기값

1	**위치**	z_k	이번 스캔 플롯
2	**속도**	(z_k − z_k-1) / T	두 플롯의 차 / 스캔 간격
3	**가속도**	0	두 점으로는 알 수 없다 (과제 조건)

오차 공분산 (축 a, b 의 3 x 3 칸마다)

$$P_{pp} = R_k$$
$$P_{pv} = P_{vp} = R_k / T$$
$$P_{vv} = (R_k + R_{k-1}) / T^2$$

위치 오차 = w_k
속도 오차 = (w_k − w_k-1) / T
가속도 행 · 열 = 0

w 는 측정 오차, R = 그 공분산 (1.5). 두 오차를 빼서 T 로 나누기 때문에 속도 분산은 두 R 의 합 / T^2 이 된다.

	x	x'	x''	y	y'	y''	z	z'	z''
x	943	9432	0	1148	1.1e4	0	5	53	0
x'	9432	1.9e5	0	1.1e4	2.3e5	0	53	-1006	0
x''	0	0	0	0	0	0	0	0	0
y	1148	1.1e4	0	1454	1.5e4	0	-4	-43	0
y'	1.1e4	2.3e5	0	1.5e4	2.9e5	0	-43	796	0
y''	0	0	0	0	0	0	0	0	0
z	5	53	0	-4	-43	0	2535	2.5e4	0
z'	53	-1006	0	-43	796	0	2.5e4	5.1e5	0
z''	0	0	0	0	0	0	0	0	0

표적 5 예 (5.8 km, T = 0.1 s). 단위 m^2, m^2/s, m^2/s^2

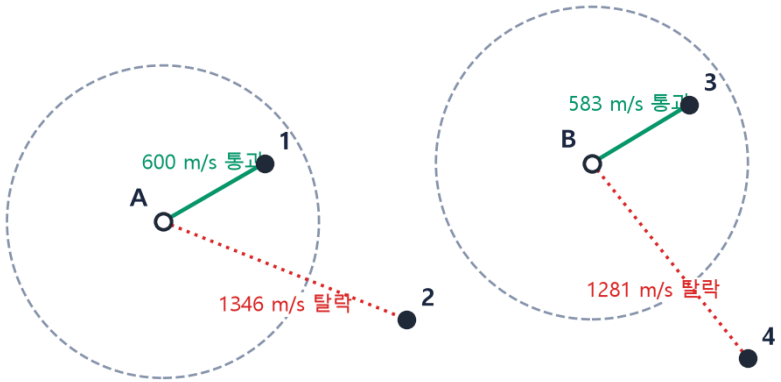
1.4 속도 게이팅

두 플롯이 같은 표적이면 한 스캔 동안 움직인 거리가 V_max · T 를 넘을 수 없습니다. 이 조건을 넘는 쌍으로만 트랙을 개시합니다.

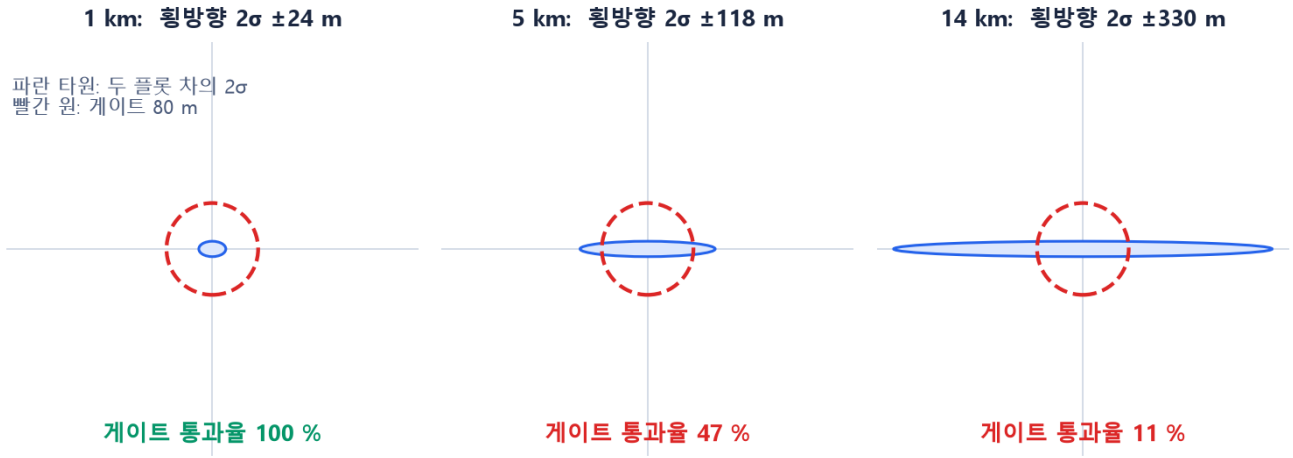
$$| z_{(k,i)} - z_{(k-1,j)} | / \Delta T < V_{max}$$

V_max = 800 m/s, ΔT = 0.1 s -> 앞 플롯에서 반경 80 m 안
i: 이번 스캔 플롯, j: 앞 스캔 플롯. 거리는 VC-NED 직교좌표로 잰다

짝짓기: 앞 스캔 플롯 (빈 원) 과 이번 스캔 플롯 (채운 원)



점선 원 = 반경 V_max · T = 80 m. 속력이 작은 쌍부터 하나씩 고르고, 쓴 플롯은 다시 쓰지 않는다



스캔 간격이 짧아 게이트가 작습니다. 가로 방향 잡음은 거리에 비례해서, 멀리 있는 표적은 참 표적 쌍도 자주 게이트를 넘지 못합니다 (4.4).

1.5 측정 공분산을 VC-NED 로 옮기기

플롯의 오차는 거리 · 방위각 · 고각에 있어서, 트랙 좌표계 (VC-NED) 의 위치 공분산 R 로 바꿔야 오차 공분산을 채울 수 있습니다.

1 **측정 표준편차 (과제 4, SNR 20)**

$\sigma_r = 4.74\ \text{m}, \quad \sigma_{az} = 0.478°, \quad \sigma_{el} = 0.494°$

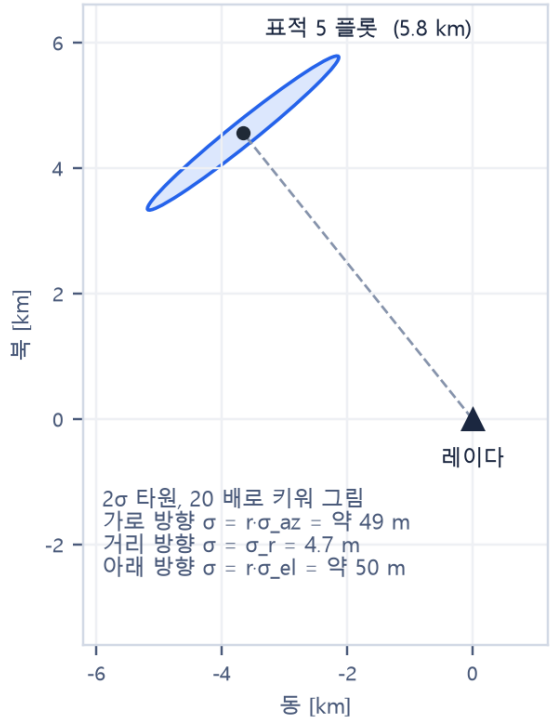
2 **안테나 직교좌표로**

$$R_{ant} = J \cdot \mathrm{diag}(\sigma_r^2, \sigma_{az}^2, \sigma_{el}^2) \cdot J^T$$

3 **VC-NED 로**

$$R = M \cdot R_{ant} \cdot M^T$$

J: 구좌표 (r, az, el) -> 안테나 직교 (x, y, z) 변환의 미분
M: 안테나 -> 동체 -> VC-NED 회전 (플랫폼 Roll · Pitch · Yaw)
회전만 들어가서 오차 타원의 크기는 그대로, 방향만 바뀐다



VC-NED 플랫폼 무게중심이 원점, 축은 북 · 동 · 아래. 자세가 바뀌어도 축이 돌지 않아 트랙을 이 좌표계에서 만든다

02. 트랙 관리

만든 트랙을 확정할지 지울지 정하는 M/N 방식과 트랙 스코어 방식을 설명합니다.

2.1 M/N 방식 10

2.2 트랙 스코어 방식 11

2.3 Assigned_Flag 에 따른 스코어 변화 12

2.1 M/N 방식

최근 N 번의 연관 결과 가운데 성공 횟수 M 으로 확정을, 연속 실패 횟수로 삭제를 정합니다.

임시 트랙 (예: M = 3, N = 5)



N = 5 번 중 M = 3 번 성공 -> 확정

M 번 연속 실패 -> 삭제

확정 트랙



연관 실패 -> 메모리 트랙

정한 시간보다 길게 이어지면 -> 삭제

1 **임시 트랙**
N 번 중 M 번 트랙 연관에 성공하면 확정
M 번 연속 결합에 실패하면 삭제

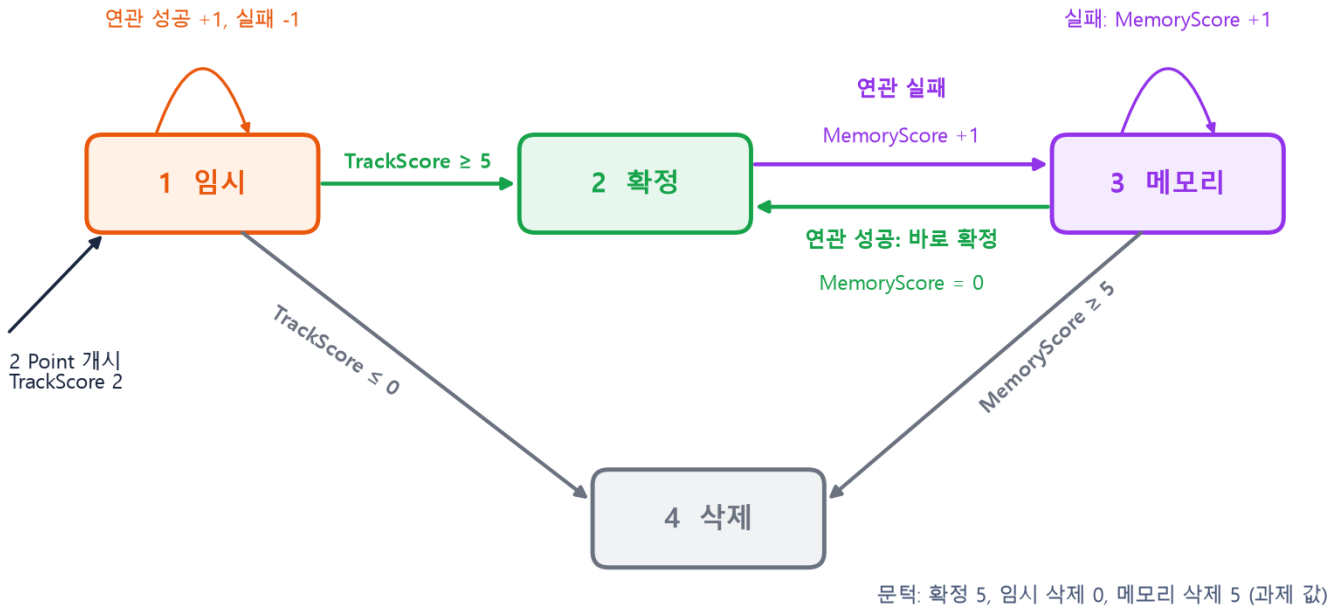
2 **확정 트랙**
트랙 연관에 실패하면 메모리 트랙으로 바꿈
정한 시간 이상 메모리 상태면 삭제

3 **특징**
성공 · 실패 횟수만 세어 단순하다
트랙 상태를 값 하나로 나타내기는 어렵다

이번 과제는 다음 쪽의 트랙 스코어 방식을 적용했습니다.

2.2 트랙 스코어 방식

트랙마다 TrackScore 와 MemoryScore 를 두고, 점수가 문턱을 넘으면 상태 (TrackState) 를 바꿉니다.



1. **TrackState (과제 값)**
   1 임시, 2 확정, 3 메모리, 4 삭제

2. **문턱 (과제 값)**
   임시 트랙 확정 5, 임시 트랙 삭제 0
   메모리 트랙 삭제 5

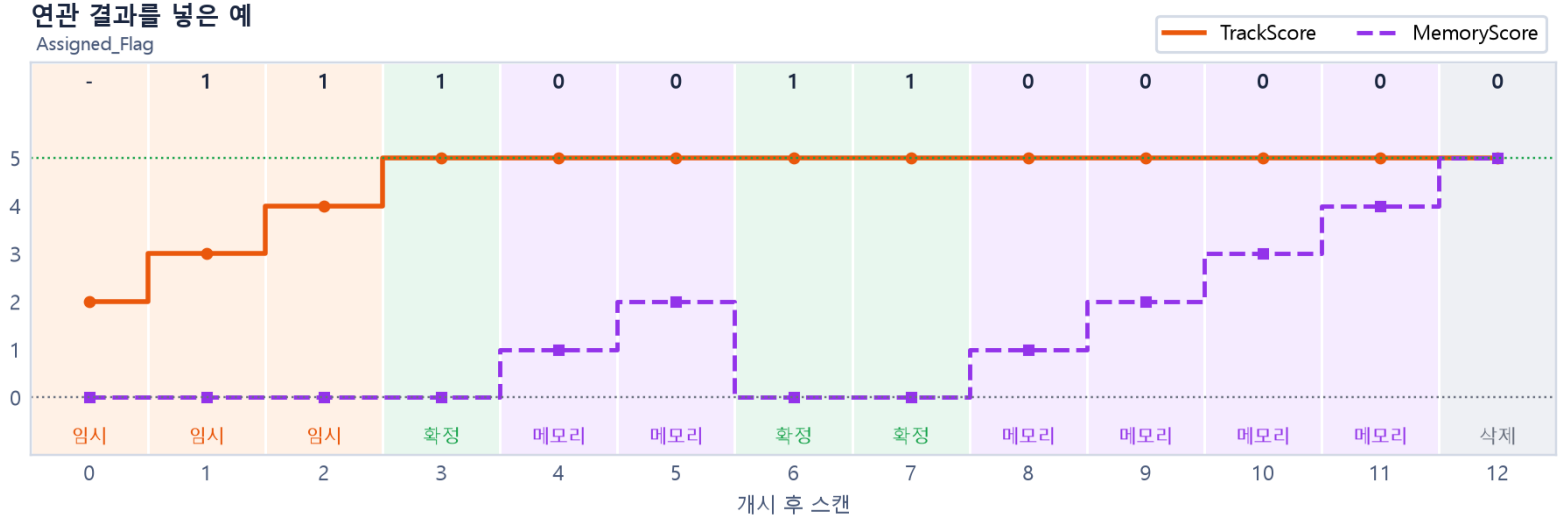
3. **메모리 트랙이 다시 연관되면**
   바로 확정 트랙으로, MemoryScore = 0

4. **과제에 없어 정한 값**
   개시 직후 TrackScore 2 (두 플롯으로 만든 트랙)
   한 번에 올리고 내리는 양 1

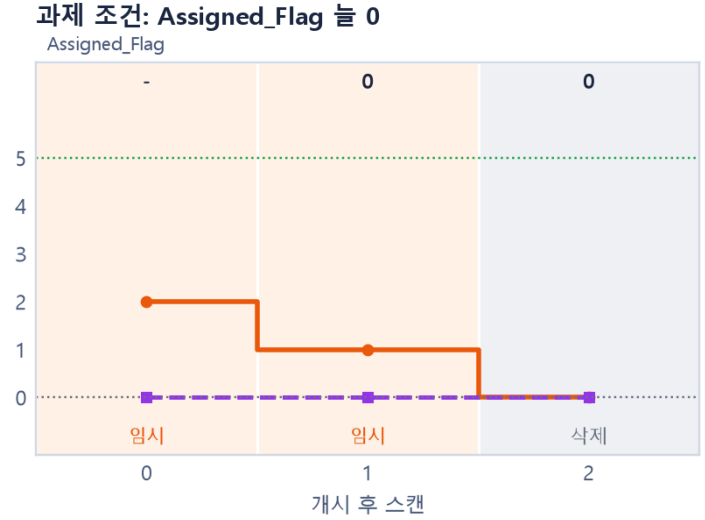
2.3 Assigned_Flag 에 따른 스코어 변화

Assigned_Flag 는 그 스캔의 트랙 연관 결과입니다 (1 성공, 0 실패). 과제 조건은 연관 기능이 없어 매 스캔 0 입니다.

연관 결과를 넣은 예



과제 조건: Assigned_Flag 늘 0



1 처리 순서
기존 트랙을 먼저 판정하고
새 트랙은 그 뒤에 개시
-> 만든 스캔에서는 감점 없음

2 과제 조건 (늘 0)
개시 스캔 2, 다음 스캔 1,
그다음 스캔 0 -> 삭제
확정 · 메모리는 나오지 않는다

3 확정 · 메모리 확인
연관 결과를 넣어 보는
스코어 확인 창으로 확인 (4.3)

03. 구현

요구사항이 어느 코드에 있는지 정리하고, 쓰레드 · UDP 구조와 트랙 개시 · 관리 코드를 봅니다.

3.1 요구사항과 구현 14

3.2 쓰레드와 UDP 구조 15

3.3 플롯 메시지 STRUCT_PlotMsg 16

3.4 송신: 플롯 메시지 만들기 (f_Lnk_BuildMsg) 17

3.5 수신: 좌표 변환과 측정 공분산 (f_Lnk_ProcessMsg) 18

3.6 2 Point 개시: 쌍 고르기 (f_Trk_ProcessScan) 19

3.7 2 Point 초기화 (f_Trk_TwoPointInit) 20

3.8 트랙 스코어 로직 (f_Trk_UpdateState) 21

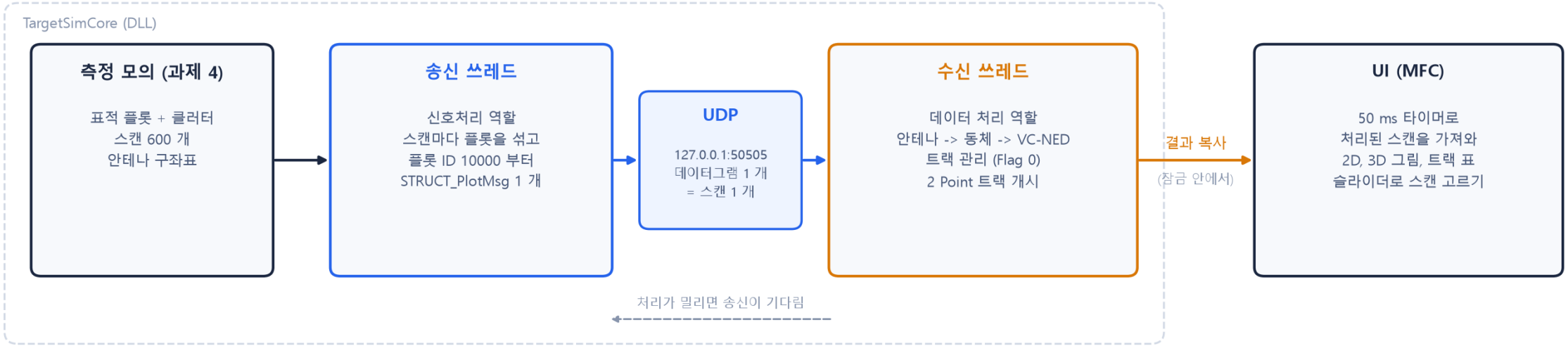
3.1 요구사항과 구현

과제 3 장 (구현) 의 항목을 구현한 곳입니다. 트랙 개시 · 관리는 TrackMgr.c, 플롯 메시지 송수신은 PlotLink.c 에 있습니다.

과제 요구사항	구현	파일	쪽
2 Point 트랙 개시, 상태변수 X_k 9 개	`f_Trk_TwoPointInit`	TrackMgr.c	20
mat_EstX, mat_EstP 가속도 항 0	Matrix_initialize 후 위치 · 속도 칸만 채움	TrackMgr.c	20
속도 게이팅 V_max = 800 m/s	`f_Trk_ProcessScan, TRK_VMAX`	TrackMgr.c	19
트랙 ID 100 ~ 10000	`f_Trk_NewId`	TrackMgr.c	19
트랙 스코어: 확정 5, 삭제 0, 메모리 삭제 5	`f_Trk_UpdateState`	TrackMgr.c	21
TrackScore, MemoryScore, TrackState (1 ~ 4)	`ST_Track, TRK_STATE_*`	TrackMgr.h	21
안테나 구좌표 플롯 메시지를 수신 쓰레드로	`STRUCT_PlotMsg, f_Lnk_TxThread / RxThread`	Idd.h, PlotLink.c	15 ~ 18
표적 플롯 + 클러터, 플롯 ID 10000 ~ 1000000	`f_Lnk_BuildMsg`	PlotLink.c	17
UDP 또는 TCP/IP 전송	UDP (과제 1 소켓 함수의 UDP 부분)	PlotLink.c	15
매 스캔 Assigned_Flag = 0 (성공 1)	f_Trk_ProcessScan 첫 단계	TrackMgr.c	19
True, Plot, Track 심볼 2D · 3D 전시	`CTrajectoryPlot, CTrackView3D`	TrajectoryPlot.cpp	23 ~ 24

3.2 쓰레드와 UDP 구조

과제 4 의 측정 모의 결과를 신호처리가 내보내는 플롯으로 보고, 송신 쓰레드가 스캔마다 UDP 로 보내 수신 쓰레드 (데이터 처리) 가 처리합니다.



1 **송신 쓰레드**
과제 1 에서 Winsock2 로 옮긴 UDP 함수
스캔 하나 = 데이터그램 하나
수신 버퍼 8 MB, 밀리면 기다림

2 **수신 쓰레드**
수신 대기는 100 ms 마다 끊어
멈춤 신호를 확인
스캔 번호로 빠진 스캔을 셈

3 **UI**
50 ms 타이머로 처리된 스캔을
가져와 그림과 표를 그림
결과는 잠금 안에서 복사

3.3 플롯 메시지 STRUCT_PlotMsg

TargetSimCore/include/Idd.h

참고 소스 코드의 Idd.h 는 STRUCT_PlotInfo 의 Y_Body 에서 끊겨 있어 나머지와 메시지 구조체를 이어 썼습니다.

플롯 하나 STRUCT_PlotInfo

```
27  typedef struct STRUCT_PlotInfo
28  {
29    UINT32  Id;
30    UINT32  EmittedDwellNum;
31    UINT32  EmittedScanNum;
32    UINT32  EmittedBeamNum;
33    UINT32  EmittedTimeStamp_s;
34    UINT32  EmittedTimeStamp_ns;
35    UCHAR8  BeamType;
36    FLOAT32  Range_Ant;
37    FLOAT32  Azimuth_Ant;
38    FLOAT32  Elevation_Ant;
39    FLOAT32  Range_VcNed;
40    FLOAT32  Azimuth_VcNed;
41    FLOAT32  Elevation_VcNed;
42    DOUBLE64  X_Ant;
43    DOUBLE64  Y_Ant;
44    DOUBLE64  Z_Ant;
45    DOUBLE64  X_Body;
46    DOUBLE64  Y_Body;
47    DOUBLE64  Z_Body;
48    DOUBLE64  X_VcNed;
49    DOUBLE64  Y_VcNed;
50    DOUBLE64  Z_VcNed;
51  }STRUCT_PlotInfo;
```

스캔 하나의 메시지 STRUCT_PlotMsg

```
62  typedef struct STRUCT_PlotMsg
63  {
64    UINT32  MsgId;
65    UINT32  MsgSize;  // 보내는 바이트 수
66    UINT32  ScanNum;
67    UINT32  TimeStamp_s;  // 스캔 시각
68    UINT32  TimeStamp_ns;
69    UINT32  PlotNum;  // 표적 플롯 + 클러터
70    FLOAT32  Sigma_Range;  // 측정 표준편차 [m]
71    FLOAT32  Sigma_Azimuth;  // [rad]
72    FLOAT32  Sigma_Elevation;  // [rad]
73    STRUCT_PlatformInfo  st_Platform;
74    STRUCT_PlotInfo  st_Plot[IDD_MAX_PLOT];
75  }STRUCT_PlotMsg;
```

1. 송신은 안테나 구좌표 세 칸만 채운다
2. Y_Body 다음부터 이어 쓴 칸 (받는 쪽이 계산해 채움)
3. 머리: 종류 · 크기 · 스캔 번호 · 시각 · 플롯 수
4. 측정 표준편차: 받는 쪽 오차 공분산 계산에 씀
5. 플랫폼 위치 · 자세, 플롯 배열 (최대 64 개)

보내는 크기 = 머리 72 B + 플롯 121 B x 플롯 수 (표적 5 + 클러터 5 -> 1,282 B)

3.4 송신: 플롯 메시지 만들기 (f_Lnk_BuildMsg)

TargetSimCore/src/PlotLink.c

표적 플롯과 클러터를 섞어 방위각 순서로 담고, 플롯마다 ID 를 매겨 안테나 구좌표만 채웁니다.

```
299  static UINT32 f_Lnk_BuildMsg(STRUCT_PlotMsg *st_Msg, INT32 nScan, UINT32 *pt_PlotId)
     ...
314    for (nIndex = 0; (nIndex < st_Scan->nTargetNum) && (nPlotNum < IDD_MAX_PLOT); nIndex++)   (1)
315    {
316      st_Sph[nPlotNum]  = st_Scan->st_Plot[nIndex];
317      nSource[nPlotNum]  = nIndex + 1;
318      nPlotNum++;
319    }
     ...
328    // 방위각 오름차순 (삽입 정렬)   (2)
     ...
346    st_Msg->MsgId  = IDD_MSG_ID_PLOT;   (3)
347    st_Msg->MsgSize  = msgSize;
348    st_Msg->ScanNum  = (UINT32)nScan;
349    st_Msg->TimeStamp_s  = (UINT32)(timeNs / 1000000000LL);
350    st_Msg->TimeStamp_ns  = (UINT32)(timeNs % 1000000000LL);
351    st_Msg->PlotNum  = (UINT32)nPlotNum;
     ...
365    for (nIndex = 0; nIndex < nPlotNum; nIndex++)
366    {
367      st_Info  = &st_Msg->st_Plot[nIndex];
368      st_Info->Id  = *pt_PlotId;   (4)
     ...
375      st_Info->Range_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].r;
376      st_Info->Azimuth_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].az;
377      st_Info->Elevation_Ant  = (FLOAT32)st_Sph[nOrder[nIndex]].el;
     ...
389      // 플롯 ID 는 10000 ~ 1000000 을 돌아가며 씀   (5)
390      *pt_PlotId = (*pt_PlotId >= IDD_PLOT_ID_MAX) ? IDD_PLOT_ID_MIN : (*pt_PlotId + 1U);
```

1. **표적 플롯 + 클러터를 한 배열에**
   클러터도 같은 모양의 반복문으로 붙임
   출처 (표적 k / 클러터) 는 송신 쪽에만 기록

2. **방위각 순서로 정렬**
   빔이 지나가는 순서처럼
   배열 위치로 표적을 알 수 없게

3. **메시지 머리**
   스캔 번호, 시각, 플롯 수

4. **플롯 ID 와 안테나 구좌표**
   거리 · 방위각 · 고각만 채움

5. **플롯 ID 10000 ~ 1000000**
   1000000 다음은 10000 으로

보내기: 송신 쓰레드가 과제 1 의 `f_SocketSendUDP_IPv4_Normal`

3.5 수신: 좌표 변환과 측정 공분산 (f_Lnk_ProcessMsg)

TargetSimCore/src/PlotLink.c

받은 플롯을 VC-NED 로 바꾸고 측정 공분산을 구한 뒤, 트랙 관리와 개시를 한 번에 돌립니다.

```
593  static VOID f_Lnk_ProcessMsg(STRUCT_PlotMsg *st_Msg)
     ...
595    const FLOAT64  time = (FLOAT64)st_Msg->TimeStamp_s + ((FLOAT64)st_Msg->TimeStamp_ns * 1.0e-9);
     ...
610    st_Sigma.r  = st_Msg->Sigma_Range;                                  ① 
611    st_Sigma.az  = st_Msg->Sigma_Azimuth;
612    st_Sigma.el  = st_Msg->Sigma_Elevation;
613    st_Att.Roll  = st_Msg->st_Platform.Pf_Roll;
614    st_Att.Pitch  = st_Msg->st_Platform.Pf_Pitch;
615    st_Att.Yaw  = st_Msg->st_Platform.Pf_Yaw;
     ...
617    for (nPlot = 0; nPlot < nPlotNum; nPlot++)
618    {
619      st_Info = &st_Msg->st_Plot[nPlot];
620      f_Lnk_FillPlotCoord(st_Info, &st_Msg->st_Platform);               ②
     ...
628      st_Plot[nPlot].pos[0]  = st_Info->X_VcNed;                        ③
629      st_Plot[nPlot].pos[1]  = st_Info->Y_VcNed;
630      st_Plot[nPlot].pos[2]  = st_Info->Z_VcNed;
     ...
632      f_Trk_MeasCov(st_Plot[nPlot].cov, &st_Meas, &st_Sigma, &st_Att);
     ...
643      f_Trk_MoveOrigin(&s_Link.st_Table, shift);                        ④
     ...
649    f_Trk_ProcessScan(&s_Link.st_Table, st_Plot, nPlotNum, time, (INT32)st_Msg->ScanNum, &st_Log);   ⑤
```

2 f_Lnk_FillPlotCoord 의 변환 세 줄

PlotLink.c

```
541  st_Ant  = f_Trans_Ant_Sph_To_XYZ(st_Info->Range_Ant, st_Info->Azimuth_Ant, st_Info->Elevation_Ant);
542  st_Body  = f_Trans_Ant_To_Body(st_Ant.x, st_Ant.y, st_Ant.z);
543  st_Ned  = f_Trans_Body_To_Ned(st_Body.x, st_Body.y, st_Body.z, st_Platform->Pf_Roll, st_Platform->Pf_Yaw, st_Platform->Pf_Pitch);
```

① **머리에서 꺼냄**
시각, 측정 표준편차, 플랫폼 자세

② **안테나 -> 동체 -> VC-NED**
아래 세 줄. 비어 있던 좌표 칸도 채움

③ **트랙 개시용 플롯**
위치 VC-NED, 공분산 f_Trk_MeasCov (1.5)

④ **원점 옮기기**
플랫폼이 움직이면 앞 스캔 플롯 · 트랙을 이번 스캔 VC-NED 로 (이번 시나리오는 0)

⑤ **트랙 관리 + 2 Point 개시**
결과는 스캔 단위로 복사해 둠
(UI 가 가져감)

3.6 2 Point 개시: 쌍 고르기 (f_Trk_ProcessScan)

TargetSimCore/src/TrackMgr.c

모든 트랙을 Assigned_Flag 0 으로 판정한 뒤, 게이트를 넘는 쌍 가운데 가장 느린 쌍부터 하나씩 골라 트랙을 만듭니다.

```
313   st_Track->Assigned_Flag  = 0;                                                    1
314   f_Trk_UpdateState(st_Track);
      ...
371     bestSpeed  = TRK_VMAX;                                                         2
      ...
379         if ((st_Table->st_PrevPlot[nPrev].isUsed == 0) && (st_Cur[nCur].isUsed == 0))
380         {
381           speed = f_Trk_PairSpeed(&st_Table->st_PrevPlot[nPrev], &st_Cur[nCur], dt);
382
383           if (speed < bestSpeed)
384           {
385             bestSpeed  = speed;
386             bestPrev  = nPrev;
387             bestCur  = nCur;
      ...
399       st_Table->st_PrevPlot[bestPrev].isUsed  = 1;                                 3
400       st_Cur[bestCur].isUsed  = 1;
      ...
402       newId = (st_Table->nTrackNum < TRK_MAX_TRACK) ? f_Trk_NewId(st_Table) : 0;   4
      ...
412         f_Trk_TwoPointInit(st_Track, &st_Table->st_PrevPlot[bestPrev], &st_Cur[bestCur]);  5
413         st_Track->TrackId  = newId;
414         st_Track->TrackState  = TRK_STATE_TENTATIVE;
415         st_Track->TrackScore  = TRK_SCORE_INIT;
      ...
420         st_Track->PlotId[0]  = st_Table->st_PrevPlot[bestPrev].PlotId;
421         st_Track->PlotId[1]  = st_Cur[bestCur].PlotId;
```

1 **트랙 관리 먼저**
연관이 없어 모든 트랙 Assigned_Flag = 0
f_Trk_UpdateState 로 판정 (3.8)

2 **속도 게이트**
bestSpeed 를 V_max 로 시작
-> 800 m/s 이상인 쌍은 고르지 않음

3 **쓴 플롯 표시**
한 플롯은 한 쌍에만

4 **트랙 ID**
100 부터 차례로, 10000 다음은 100
쓰는 중인 ID 는 건너뜀

5 **초기화 (3.7)**
임시 (1), TrackScore 2, 개시 플롯 ID 2 개

3.7 2 Point 초기화 (f_Trk_TwoPointInit)

TargetSimCore/src/TrackMgr.c

9 x 1 상태변수와 9 x 9 오차 공분산을 0 으로 만든 뒤, 축마다 위치 · 속도 칸만 채웁니다. 가속도 칸은 0 으로 남습니다.

```
164 VOID f_Trk_TwoPointInit(ST_Track *st_Track, const ST_TrkPlot *st_Prev, const ST_TrkPlot *st_Cur)
165 {
166   const FLOAT64  dt = st_Cur->time - st_Prev->time;
167   matrix  *st_X = &st_Track->mat_EstX;
168   matrix  *st_P = &st_Track->mat_EstP;
169   INT32  nAxis;
170   INT32  nAxisB;
171
172   Matrix_initialize(st_X, TRK_STATE_DIM, 1);
173   Matrix_initialize(st_P, TRK_STATE_DIM, TRK_STATE_DIM);
174
175   for (nAxis = 0; nAxis < 3; nAxis++)
176   {
177     st_X->e[(3 * nAxis)][0]  = st_Cur->pos[nAxis];
178     st_X->e[(3 * nAxis) + 1][0]  = (st_Cur->pos[nAxis] - st_Prev->pos[nAxis]) / dt;
179     st_X->e[(3 * nAxis) + 2][0]  = 0.0;
180
181     for (nAxisB = 0; nAxisB < 3; nAxisB++)
182     {
183       st_P->e[(3 * nAxis)][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB];
184       st_P->e[(3 * nAxis)][(3 * nAxisB) + 1]  = st_Cur->cov[nAxis][nAxisB] / dt;
185       st_P->e[(3 * nAxis) + 1][(3 * nAxisB)]  = st_Cur->cov[nAxis][nAxisB] / dt;
186       st_P->e[(3 * nAxis) + 1][(3 * nAxisB) + 1]  = (st_Cur->cov[nAxis][nAxisB] + st_Prev->cov[nAxis][nAxisB]) / (dt * dt);
187     }
188   }
189 }
```

1

2

3

1 0 으로 채운 행렬
참고 소스 코드의 Matrix_initialize
가속도 칸은 손대지 않아 0

2 상태변수 (축마다)
위치 = 이번 플롯
속도 = 위치 차 / 시간 차, 가속도 = 0

3 오차 공분산 (축 a, b)
P_pp = R_k, P_pv = P_vp = R_k / T
P_vv = (R_k + R_k-1) / T²
축 사이 칸까지 같은 식 (1.3)

인덱스
축 a 의 위치 3a, 속도 3a+1, 가속도 3a+2
x 북 0 ~ 2, y 동 3 ~ 5, z 아래 6 ~ 8

3.8 트랙 스코어 로직 (f_Trk_UpdateState)

TargetSimCore/src/TrackMgr.c

상태마다 case 하나씩입니다. 임시는 TrackScore 로, 확정 · 메모리는 MemoryScore 로 다음 상태를 정합니다.

임시 트랙

```
108 VOID f_Trk_UpdateState(ST_Track *st_Track)
109 {
110   const INT32 isAssigned = (st_Track->Assigned_Flag != 0) ? 1 : 0;
111
112   switch (st_Track->TrackState)
113   {
114   case TRK_STATE_TENTATIVE:
115     st_Track->TrackScore = st_Track->TrackScore + ((isAssigned != 0) ? 1 : -1);
116
117     if (st_Track->TrackScore >= TRK_SCORE_CONFIRM)
118     {
119       st_Track->TrackState = TRK_STATE_CONFIRMED;
120     }
121     else if (st_Track->TrackScore <= TRK_SCORE_DELETE)
122     {
123       st_Track->TrackState = TRK_STATE_DELETED;
124     }
125     else
126     {
127       // 임시 트랙 유지
128     }
129     break;
130
```

확정 트랙, 메모리 트랙

```
131   case TRK_STATE_CONFIRMED:
132     if (isAssigned == 0)
133     {
134       st_Track->TrackState  = TRK_STATE_MEMORY;
135       st_Track->MemoryScore  = st_Track->MemoryScore + 1;
136     }
137     break;
138
139   case TRK_STATE_MEMORY:
140     if (isAssigned != 0)
141     {
142       st_Track->TrackState  = TRK_STATE_CONFIRMED;
143       st_Track->MemoryScore  = 0;
144     }
145     else
146     {
147       st_Track->MemoryScore  = st_Track->MemoryScore + 1;
148
149       if (st_Track->MemoryScore >= TRK_MEMORY_DELETE)
150       {
151         st_Track->TrackState = TRK_STATE_DELETED;
152       }
153     }
154     break;
155
```

1. 성공 +1, 실패 -1
2. 5 이상 확정, 0 이하 삭제
3. 확정 -> 메모리, MemoryScore +1
4. 메모리 -> 바로 확정, MemoryScore 0
5. MemoryScore 5 이상 삭제

04. 결과

프로그램 화면과 송수신 결과, 2D · 3D 전시, 트랙 수명, 거리에 따른 개시 결과를 봅니다.

4.1 실행 화면과 송수신 결과 23

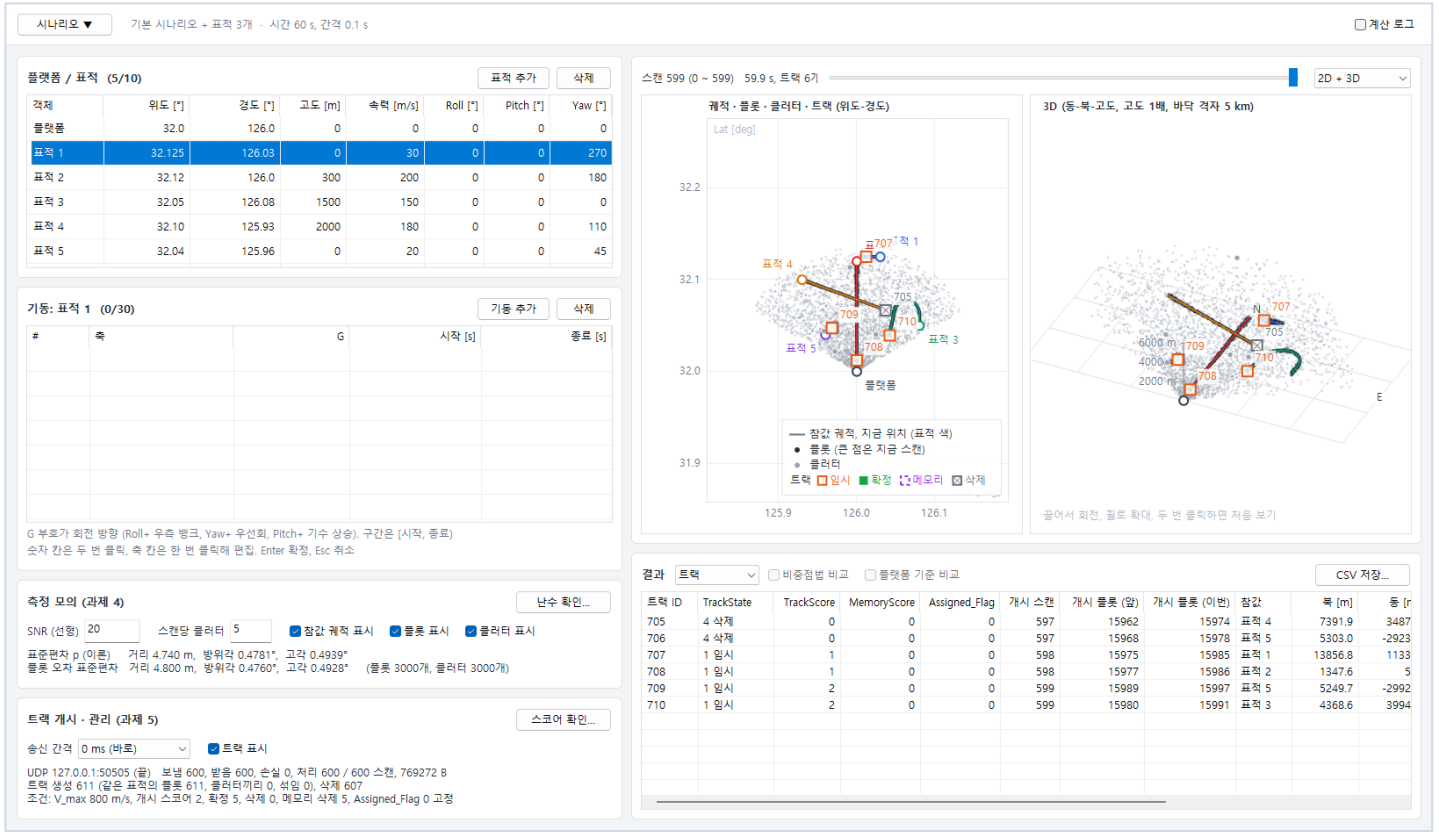
4.2 True · Plot · Track 2D · 3D 전시 24

4.3 트랙 수명과 스코어 확인 25

4.4 거리에 따른 개시 결과 26

4.1 실행 화면과 송수신 결과

기본 시나리오 + 표적 3 개, 60 s, 스캔 600 개, 스캔당 클러터 5 개 (과제 4 조건). 실행하면 바로 송수신을 시작합니다.



1 **UDP 송수신**
600 스캔 보냄, 600 스캔 받음, 손실 0
메시지 1,282 B x 600 = 769,200 B

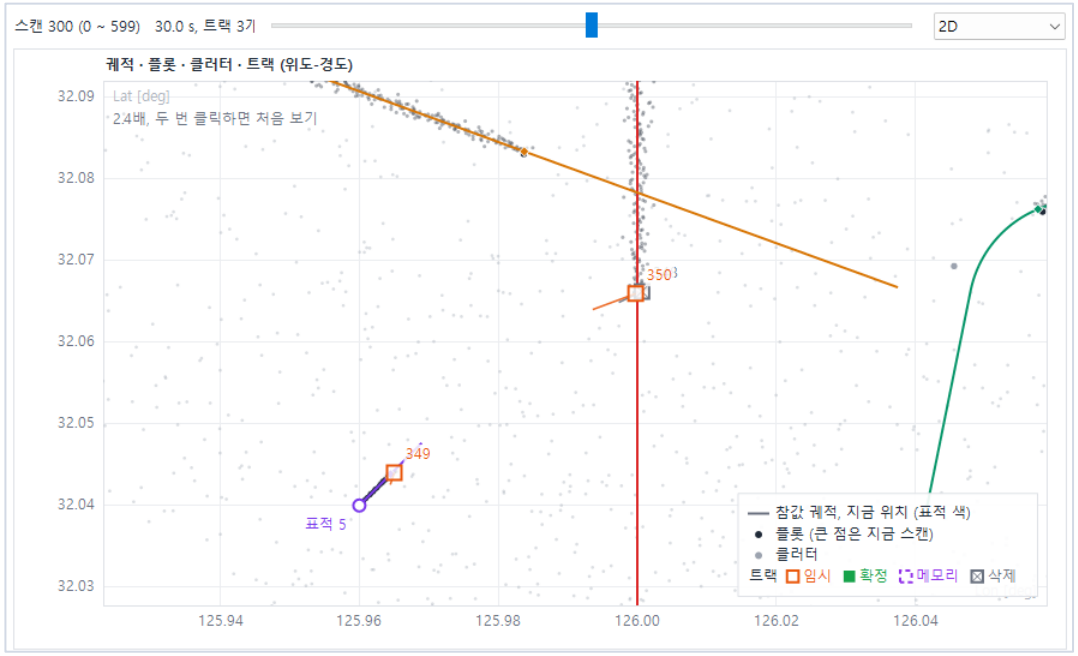
2 **트랙 611 개 개시, 607 개 삭제**
개시 쌍은 모두 같은 표적의 플롯
클러터끼리 0, 섞임 0 (송신 쪽 기록과 대조)

3 **스캔 슬라이더**
고른 스캔의 2D, 3D 그림과 트랙 표
송신 간격 100 ms 면 실시간으로 따라감

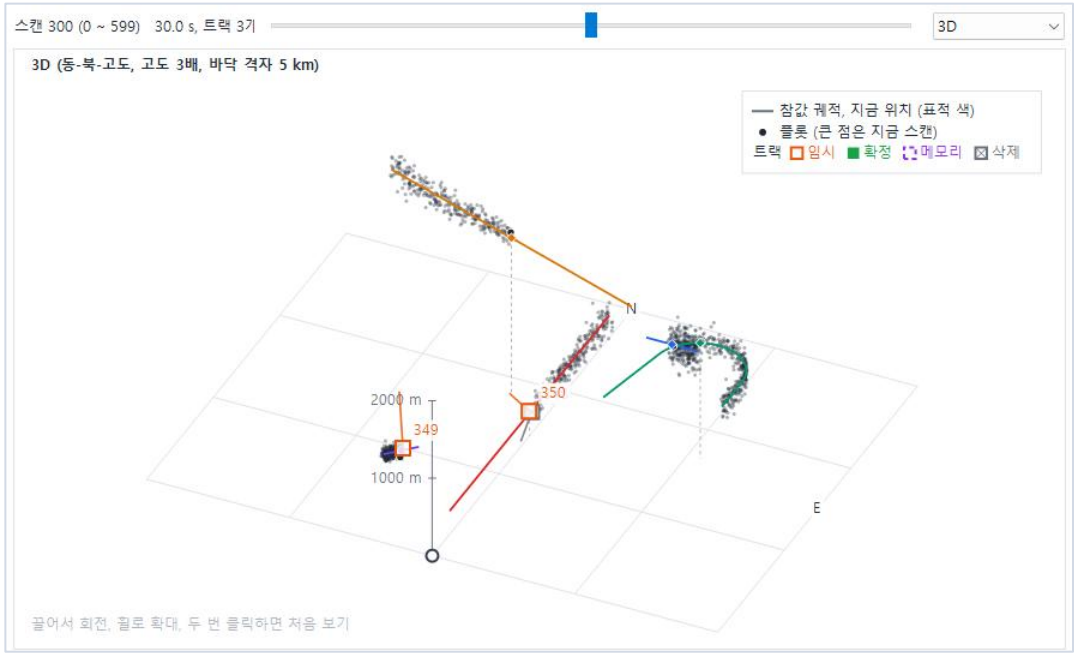
4 **트랙 표**
트랙 ID, TrackState, TrackScore,
MemoryScore, Assigned_Flag, 위치 · 속도

4.2 True · Plot · Track 2D · 3D 전시

같은 스캔 (300, 30 s) 을 2D 와 3D 로 그린 화면입니다. 참값, 플롯, 트랙 심볼을 함께 그립니다.



2D (위도 · 경도), 휠로 2.4 배 확대



3D, 클러터를 끄고 고도 3 배

참값 선: 궤적, 마름모: 지금 위치 (표적 색)

트랙 빈 주황 사각형: 임시, 회색 X: 삭제, 숫자: ID

플롯 옅은 점: 지난 스캔, 흰 테 큰 점: 지금 스캔

속도 선 트랙 속도로 1 초 동안 가는 거리

4.3 트랙 수명과 스코어 확인

과제 조건 (Assigned_Flag 늘 0) 에서 트랙은 세 스캔 동안만 보입니다. 확정 · 메모리 상태는 스코어 확인 창으로 확인했습니다.

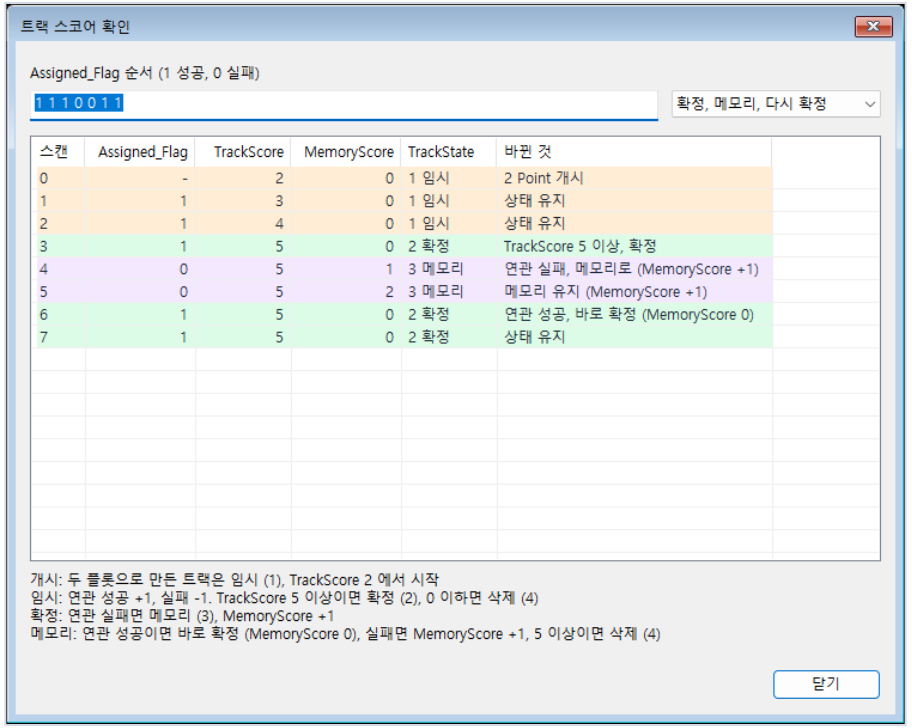
스캔 300 의 트랙 표

결과 트랙 ▾ ☐ 비중점법 비교 ☐ 플랫폼 기준 비교 CSV 저장...

트랙 ID	TrackState	TrackScore	MemoryScore	Assigned_Flag	개시 스캔	개시 플롯 (앞)	개시 플롯 (이번)	참값	북 [m]	동 [r
348	4 삭제	0	0	0	298	12974	12985	표적 2	7344.5	62
349	1 임시	1	0	0	299	12989	12999	표적 5	4881.5	-3301
350	1 임시	2	0	0	300	12997	13005	표적 2	7315.0	-23

1 **나이가 다른 트랙 세 개**
350: 이번 스캔에 개시, TrackScore 2
349: 한 스캔 전 개시, 1 / 348: 두 스캔 전 개시, 0 -> 삭제 (4)

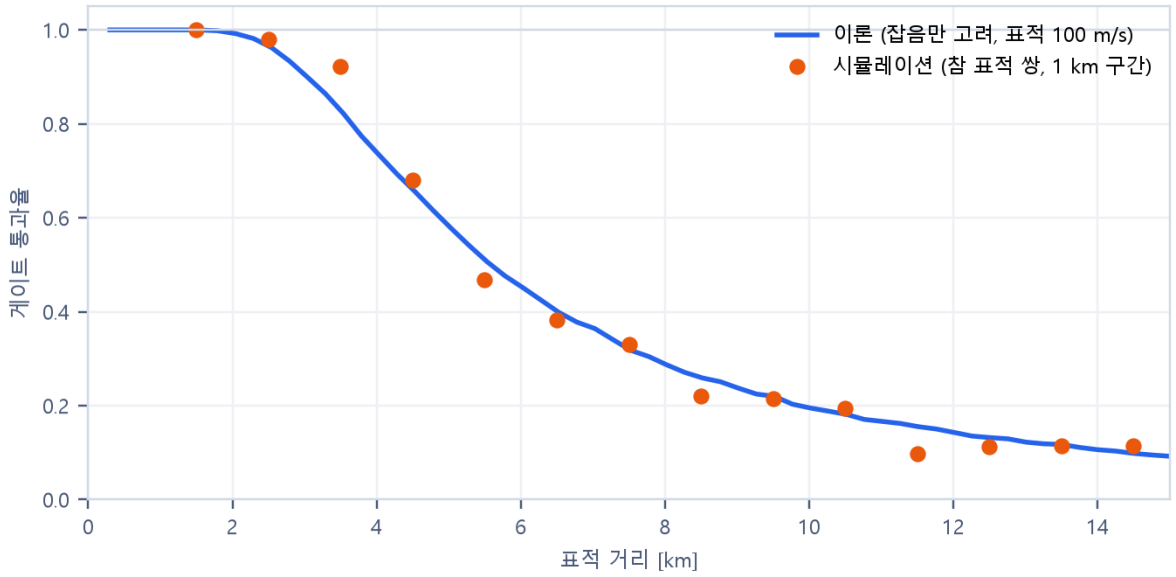
2 **모든 트랙이 같은 수명**
개시 2 -> 1 -> 0 (삭제), 다음 스캔에 표에서 빠짐
611 개 모두 이 순서 (확인: 스캔마다 TrackScore = 2 - 나이)



스코어 확인 창: 1 1 1 0 0 1 1 을 넣은 결과

4.4 거리에 따른 개시 결과

각도 오차는 거리에 비례해 위치 오차가 됩니다. 스캔 간격 0.1 s 에서 게이트 반경은 80 m 라 먼 표적일수록 개시가 드뭅니다.

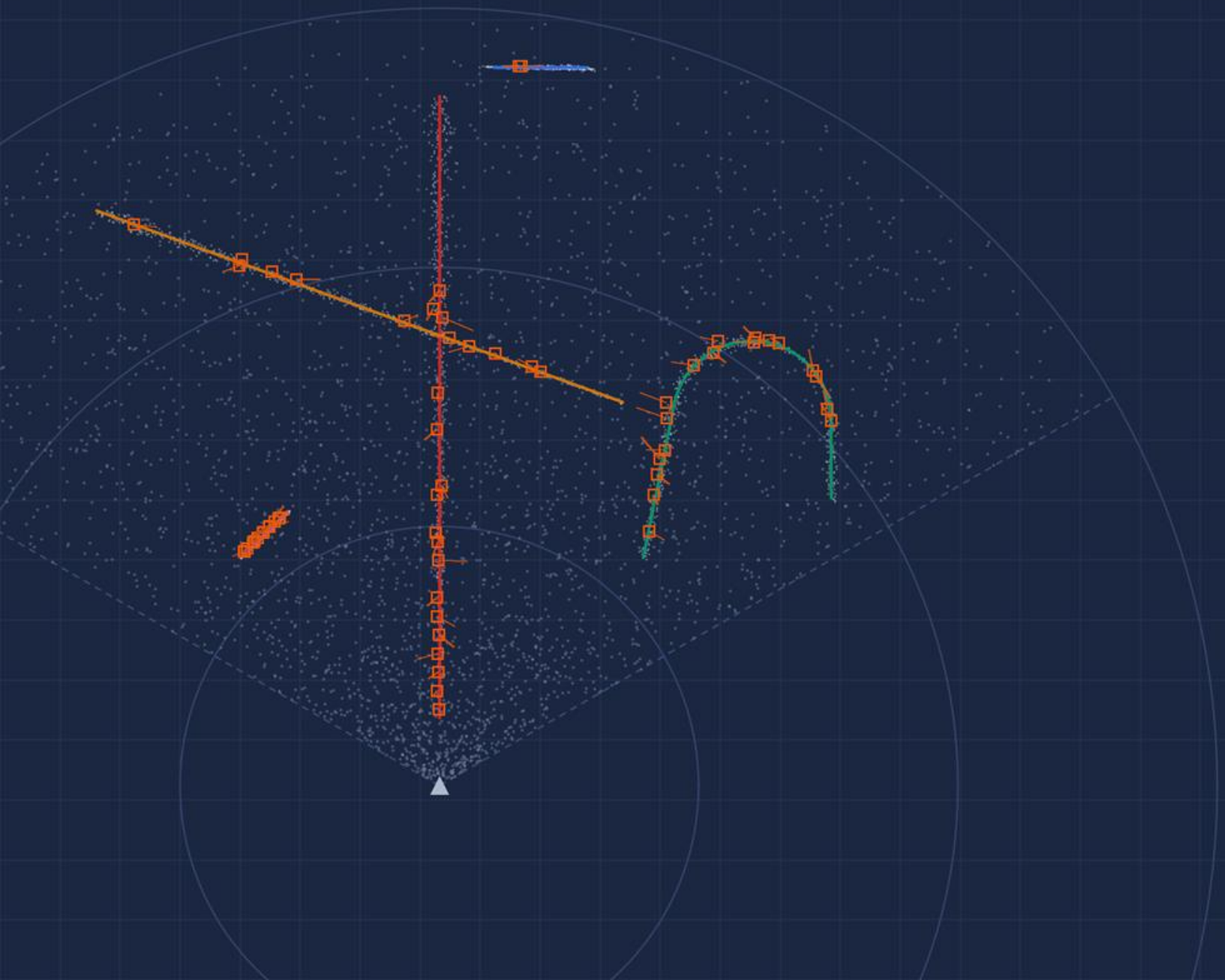


표적	거리 [km]	게이트 통과	개시 트랙	속도 오차 RMS
1	13.9 ~ 14.1	11 %	58	554 m/s
2	13.3 -> 1.4	43 %	158	475 m/s
3	6.1 ~ 10.9	25 %	115	575 m/s
4	8.4 ~ 13.1	20 %	98	531 m/s
5	5.8 ~ 6.0	45 %	182	549 m/s

게이트 통과: 참 표적 쌍 기준. 개시 트랙: 한 번 쓴 플롯은 다시 쓰지 않아 최대 300. 속도 오차: 개시 순간 추정 속도와 참 속도 차

속도 오차가 큰 이유
위치 오차를 0.1 s 로 나누기 때문. 스캔 간격이 수 초인 탐색 레이다라면
같은 위치 오차에서 속도 오차가 수십 분의 일로 준다.

1 **SNR 1000 이면**
모든 표적이 두 스캔마다 개시 (각 300 개)
정규화 속도 오차 제곱 평균 2.8 ~ 2.9 (이론 3) -> P 가 실제 오차와 맞다



감사합니다

질문 주시면 답변드리겠습니다